
UCSD ECE 226 – Optimization and Acceleration of Deep Learning on Various Hardware Platforms

# KV Cache Implementation for a Transformer Decoder

Atharvraj Patil, Rishabh Thapliyal, Lakshya Goyal
Date: 03/19/2025

Instructor:
Prof. Farinaz Koushanfar
farinaz@ucsd.edu

UC San Diego
JACOBS SCHOOL OF ENGINEERING
Electrical and Computer Engineering

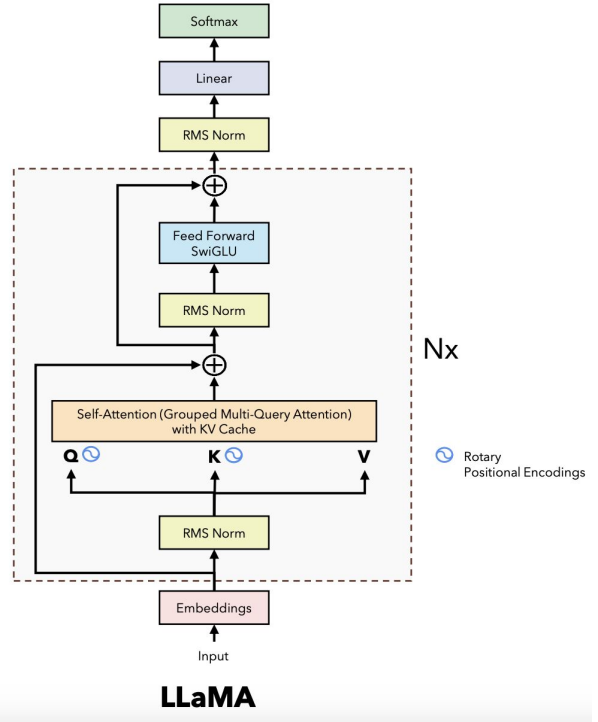
# Problem Statement

## Problem

Transformer models are widely used for text generation tasks, but their inference latency and memory usage can be high due to repeated computations in the attention mechanism.

## Solution

KV (Key-Value) caching is a technique to optimize inference by storing and reusing intermediate computations.



# Objective

***How does implementing a KV cache in a Transformer decoder affect end-to-end inference latency and memory usage??***

**Methodology:**

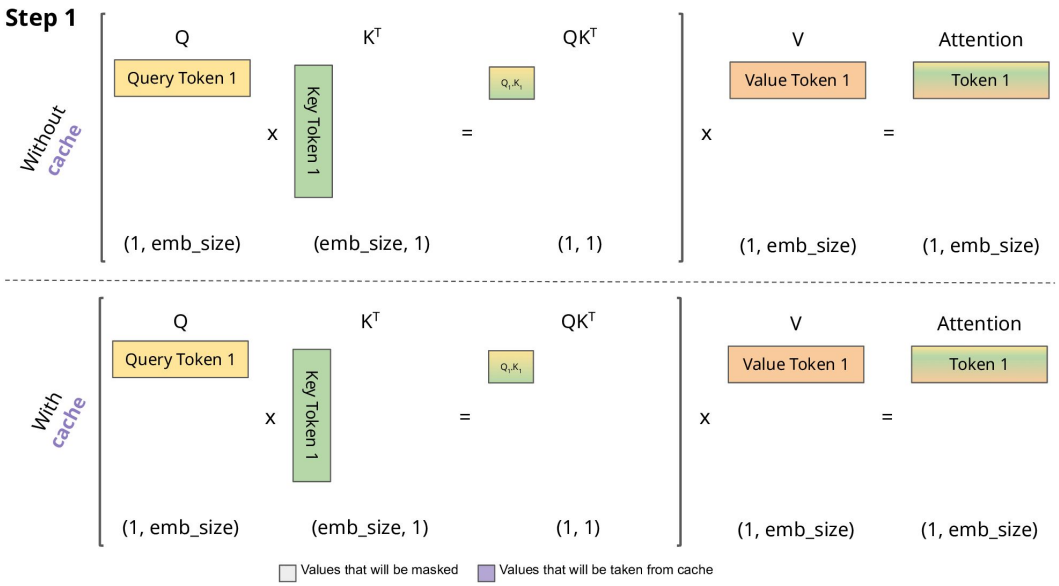
1. Train a Transformer decoder on the Penn Treebank (PTB) dataset for text completion.
2. Implement a KV cache within the attention mechanism.
3. Compare inference latency and memory usage with and without the KV cache.

**Reference:**

- Inspired by the LLaMA repository's KV cache implementation for decoder-only Transformers.
- Focus on adapting the KV cache for self-attention in a custom Transformer decoder.

# What is KV Cache?

- Instead of computing the entire K and V matrices every time, we just compute the K and V **vectors** for the input token and append it to our cache K and cache V matrices.
- The new token's arrival does not alter the key (K) and value (V) representations of the previous tokens, allowing us to reuse their precomputed values.
- Attention scores are computed for the new query token using the updated cached K and V matrices.



# Prior Art, Challenges, and Alternatives to KV Cache


**Prior Art:** KV Cache is used in GPT, T5, and LLaMA for efficient autoregressive decoding.

| Technique | Complexity | Memory Usage | Advantages | Limitations |
|---|---|---|---|---|
| **KV Cache** | O(n) | High | Reduces redundant computation; speeds up autoregressive decoding. | High memory overhead; struggles with very long sequences. |
| **Sparse Attention** | O(nlogn) | Moderate | Efficient for long sequences; reduces computation. | May lose global context; complex to implement. |
| **Quantization/Pruning** | O(n) | Low | Significant memory and computation savings; easier deployment on edge devices. | May lead to accuracy loss if not done carefully. |
| **Hybrid Approaches** | Varies | Moderate | Balances memory, computation, and accuracy; combines strengths of techniques. | Increased implementation complexity. |

# Experimental Setup

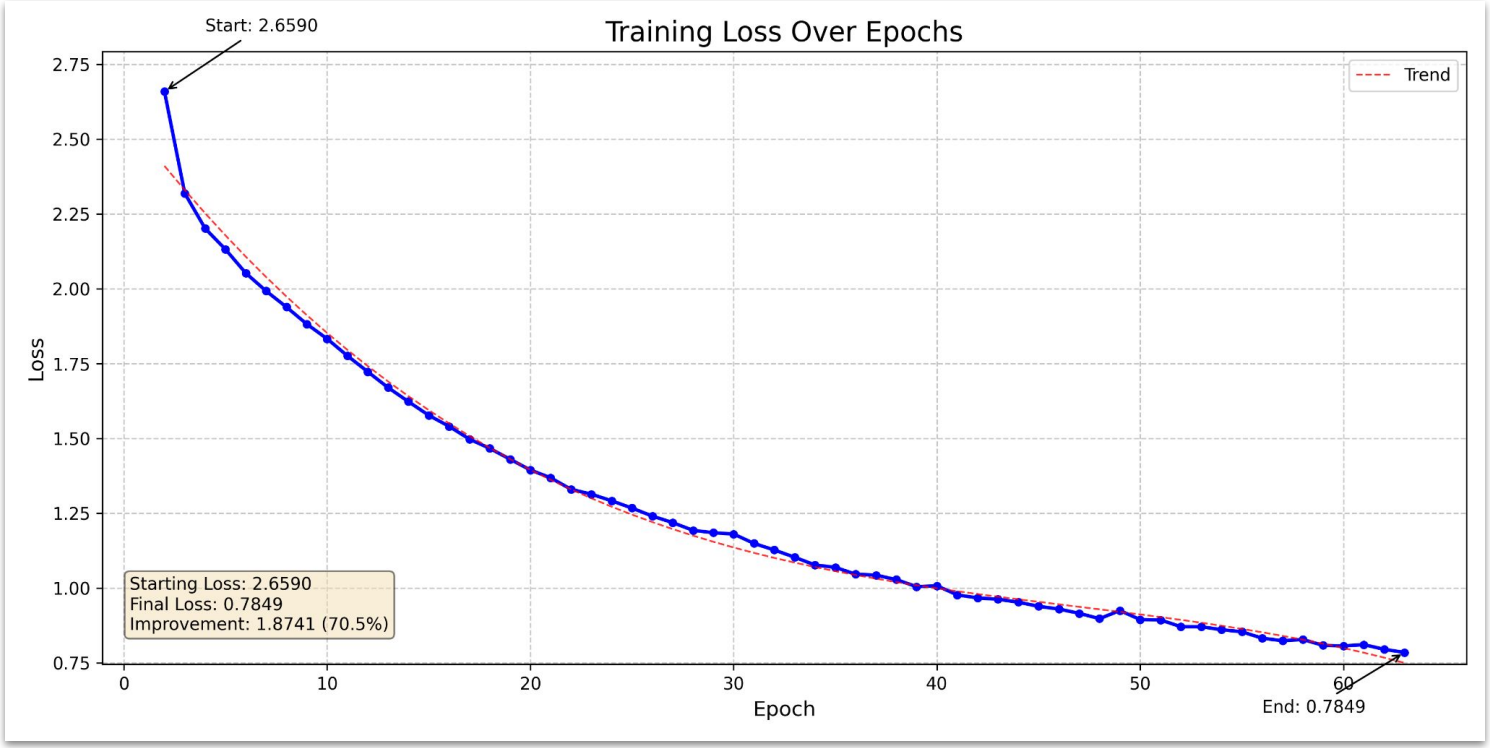
- Used PyTorch built in tokenizer for general english.
- We are training the embedding during the training process.
- Training Parameters are:

| Training Parameters | Value |
|---|---|
| Batch Size | 32 |
| Max Sequence Length | 2048 |
| Number of transformer layers | 2 |
| Number of KV Heads | 2 |
| Vocab Size | 9925 |

```
if not self.is_training and self.is_kv_cache:  # Inference mode (use KV cache)
    # Update KV cache
    self.cache_k = self.cache_k.to(xq)  # Ensure cache is on the same device as xq
    self.cache_v = self.cache_v.to(xq)
    self.cache_k[:bsz, start_pos: start_pos + seqlen] = xk
    self.cache_v[:bsz, start_pos: start_pos + seqlen] = xv

    # Retrieve cached keys and values
    keys = self.cache_k[:bsz, : start_pos + seqlen]
    values = self.cache_v[:bsz, : start_pos + seqlen]
else:  # Training mode (no KV cache)
    keys = xk
    values = xv
```

# Training


Training Loss Over Epochs
Start: 2.6590
Trend
Loss
Epoch
Starting Loss: 2.6590
Final Loss: 0.7849
Improvement: 1.8741 (70.5%)
End: 0.7849
2.75
2.50
2.25
2.00
1.75
1.50
1.25
1.00
0.75
0
10
20
30
40
50
60

# Validation Results

## Example 1

Input Text: <bos> <unk> used one to give away the house that rock star jon <unk> <unk> grew up in <pad> <pad> <pad> <pad> <pad> <pad> <pad> <pad> <pad> <pad> <pad> <pad> <pad> <pad> <pad> <pad> <pad> <pad> <pad> <pad> <pad> <pad> <pad> <pad> <pad>

Generated Text: <bos> <unk> used one to give away the house that rock star jon <unk> <unk> grew up in a few blocks away at the u . s . ambassador ' s residence the guards <unk> the compound also had discarded their <unk> arms for the bank ' s international investment <unk> that safety firm said

Generated Text: <bos> <unk> used one to give away the house that rock star jon <unk> <unk> grew up in a few blocks away at the u . s . ambassador ' s residence the guards <unk> the compound also had discarded their <unk> arms for the bank ' s international investment <unk> that safety firm said

## Example 2

Input Text: <bos> if the caller stays on the line and leaves a name and address for the sponsor coupons and a newsletter will be <unk> and the sponsor will be able to gather a list of desirable potential customers <pad> <pad> <pad> <pad> <pad> <pad> <pad> <pad> <pad> <pad> <pad> <pad> <pad> <pad> <pad> <pad> <pad> <pad> <pad> <pad>

Generated Text: <bos> if the caller stays on the line and leaves a name and address for the sponsor coupons and a newsletter will be <unk> and the sponsor will be able to gather a list of desirable potential customers are <unk> evans recruited and that ' s true

Generated Text: <bos> if the caller stays on the line and leaves a name and address for the sponsor coupons and a newsletter will be <unk> and the sponsor will be able to gather a list of desirable potential customers are <unk> evans recruited and that ' s true

*Generated Text With KV cache* = *Generated Text Without KV cache*

# Inference (using KV cache)


| Name | Self CPU % | Self CPU | CPU total % | CPU total | CPU time avg | Self CUDA | Self CUDA % | CUDA total | CUDA time avg | CPU Mem | Self CPU Mem |
|---|---|---|---|---|---|---|---|---|---|---|---|
| aten::matmul | 6.97% | 98.517ms | 21.36% | 302.154ms | 94.660us | 73.060ms | 4.95% | 336.237ms | 105.337us | 0 b | 0 b |
| CUDA Mem | Self CUDA Mem | # of Calls | Total MFLOPs | | | | | | | | |
| 628.45 Mb | -1.82 Gb | 3192 | -- | | | | | | | | |
| aten::linear | 2.48% | 35.006ms | 19.21% | 271.644ms | 107.795us | 32.657ms | 2.21% | 297.809ms | 118.178us | 0 b | 0 b |
| 508.93 Mb | 0 b | 2520 | -- | | | | | | | | |
| aten::embedding | 0.27% | 3.768ms | 13.77% | 194.755ms | 1.159ms | 3.742ms | 0.25% | 195.560ms | 1.164ms | 0 b | 0 b |
| 10.50 Mb | 0 b | 168 | -- | | | | | | | | |
| aten::unbind | 5.22% | 73.880ms | 13.64% | 192.857ms | 634.398us | 57.360ms | 3.89% | 193.851ms | 637.668us | 0 b | 0 b |
| 0 b | 0 b | 304 | -- | | | | | | | | |
| aten::index_select | 0.29% | 4.112ms | 13.15% | 185.997ms | 1.107ms | 184.133ms | 12.48% | 186.855ms | 1.112ms | 0 b | 0 b |
| 10.50 Mb | 0 b | 168 | -- | | | | | | | | |
| aten::select | 5.56% | 78.633ms | 7.45% | 105.318ms | 8.182us | 107.962ms | 7.32% | 148.508ms | 11.537us | 0 b | 0 b |
| 0 b | 0 b | 12872 | -- | | | | | | | | |
| aten::reshape | 3.06% | 43.238ms | 6.63% | 93.763ms | 19.245us | 47.935ms | 3.25% | 118.023ms | 24.225us | 0 b | 0 b |
| 1.91 Gb | 0 b | 4872 | -- | | | | | | | | |
| aten::mm | 2.47% | 34.939ms | 4.11% | 58.058ms | 23.039us | 109.672ms | 7.44% | 109.672ms | 43.521us | 0 b | 0 b |
| 508.93 Mb | 508.93 Mb | 2520 | 127920.243 | | | | | | | | |
| aten::item | 2.94% | 41.510ms | 4.84% | 68.428ms | 9.285us | 55.129ms | 3.74% | 90.348ms | 12.259us | 0 b | 0 b |
| 0 b | 0 b | 7370 | -- | | | | | | | | |
| aten::as_strided | 0.38% | 5.433ms | 0.38% | 5.433ms | 0.254us | 78.608ms | 5.33% | 78.608ms | 3.673us | 0 b | 0 b |
| 0 b | 0 b | 21404 | -- | | | | | | | | |

Self CPU time total: 1.414s
Self CUDA time total: 1.475s

*For 2 batches, each of size 32*

# Inference (w/o KV cache)


| CUDA time avg | CPU Mem | Self CPU Mem | Name<br>CUDA Mem | Self CPU %<br>Self CUDA Mem | Self CPU<br># of Calls | CPU total %<br>Total MFLOPs | CPU total | CPU time avg | Self CUDA | Self CUDA % | CUDA total |
|---|---|---|---|---|---|---|---|---|---|---|---|
| | | | aten::matmul | 6.06% | 179.811ms | 22.80% | 676.593ms | 211.965us | 109.824ms | 3.37% | 1.221s |
| 382.525us | 0 b | 0 b | 20.79 Gb | -2.76 Gb | 3192 | -- | | | | | |
| | | | aten::linear | 2.09% | 62.143ms | 20.32% | 602.841ms | 239.223us | 42.945ms | 1.32% | 1.076s |
| 426.819us | 0 b | 0 b | 19.48 Gb | 0 b | 2520 | -- | | | | | |
| | | | aten::mm | 2.77% | 82.247ms | 5.37% | 159.289ms | 63.210us | 796.173ms | 24.47% | 796.173ms |
| 315.942us | 0 b | 0 b | 19.48 Gb | 19.48 Gb | 2520 | 5805142.442 | | | | | |
| | | | aten::embedding | 0.24% | 7.054ms | 16.31% | 483.889ms | 2.880ms | 5.261ms | 0.16% | 485.945ms |
| 2.893ms | 0 b | 0 b | 489.27 Mb | -1.88 Mb | 168 | -- | | | | | |
| | | | aten::index_select | 0.23% | 6.733ms | 15.27% | 453.094ms | 2.697ms | 453.225ms | 13.93% | 457.879ms |
| 2.725ms | 0 b | 0 b | 489.27 Mb | 0 b | 168 | -- | | | | | |
| | | | aten::unbind | 4.03% | 119.571ms | 11.57% | 343.179ms | 1.129ms | 79.533ms | 2.44% | 272.411ms |
| 896.089us | 0 b | 0 b | 0 b | 0 b | 304 | -- | | | | | |
| | | | aten::reshape | 2.99% | 88.853ms | 7.73% | 229.470ms | 47.100us | 86.956ms | 2.67% | 237.649ms |
| 48.779us | 0 b | 0 b | 2.84 Gb | 0 b | 4872 | -- | | | | | |
| | | | aten::select | 4.81% | 142.625ms | 6.61% | 196.142ms | 15.238us | 149.972ms | 4.61% | 200.753ms |
| 15.596us | 0 b | 0 b | 0 b | 0 b | 12872 | -- | | | | | |
| | | | aten::item | 2.61% | 77.396ms | 6.21% | 184.255ms | 25.001us | 101.119ms | 3.11% | 161.565ms |
| 21.922us | 0 b | 0 b | 0 b | 0 b | 7370 | -- | | | | | |
| | | | aten::clone | 1.24% | 36.895ms | 4.31% | 127.795ms | 85.539us | 34.893ms | 1.07% | 145.081ms |
| 97.109us | 0 b | 0 b | 3.77 Gb | 0 b | 1494 | -- | | | | | |

Self CPU time total: 2.967s
Self CUDA time total: 3.254s

*For 2 batches, each of size 32*

# Comparison:

| Model | Atten: matmul Memory Usage (GB) | Inference Time (CUDA, in secs) | Inference Time (CPU, in secs) |
|---|---|---|---|
| With KV Cache | 0.62845 | 1.475 | 1.414 |
| Without KV Cache | 20.79 | 3.254 | 2.967 |

Inference time is: Time for complete text generation for 2 batches

1. **Efficiency with KV Cache:** Utilizing KV Cache significantly reduces memory usage from **20.79 GB to 628.45 MB**, making inference more efficient and enabling deployment on resource-constrained environments.

2. **Performance Improvement:** With KV Cache, CUDA inference time is reduced from **3.254 seconds to 1.475 seconds**, demonstrating a **2.2x speedup**, which enhances real-time processing capabilities.

# Evaluation Scores setup

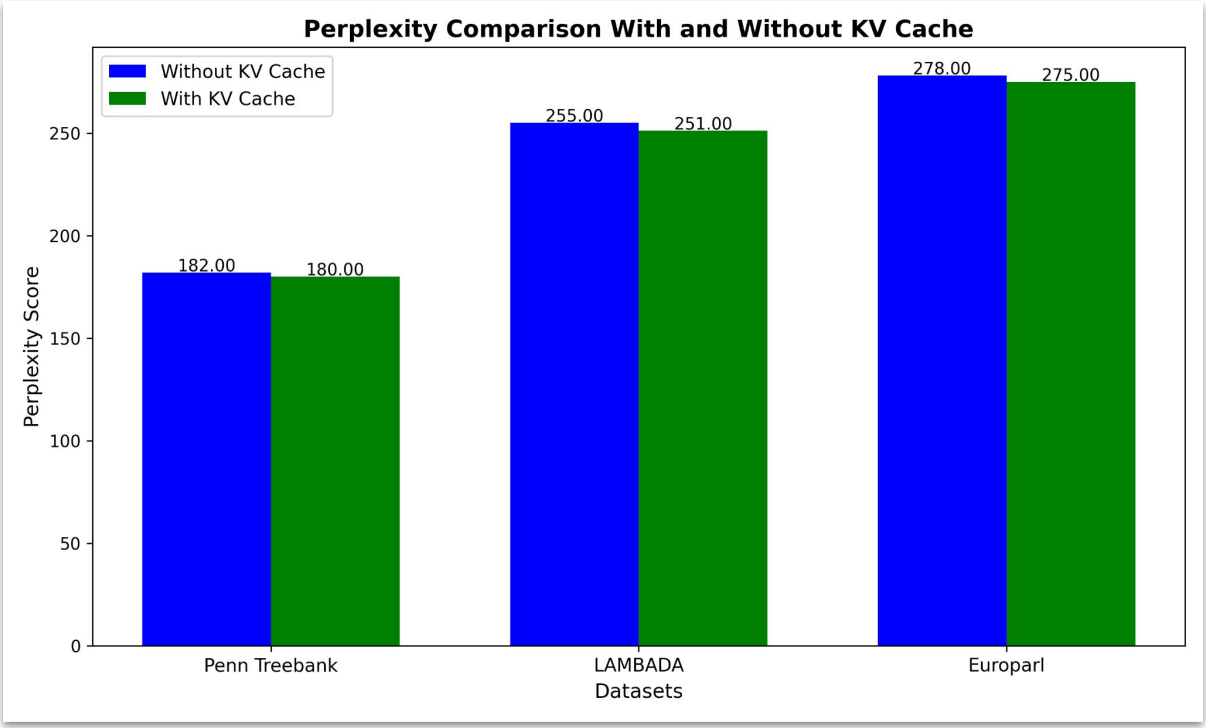
- **LAMBDA OpenAI Dataset**: Contains 5,153 English sentences for language modeling tasks.
- **Penn Treebank Dataset**: Includes 49,208 sentences primarily from the Wall Street Journal, used for syntactic parsing and linguistic research.
- **Europarl English Dataset**: Comprises approximately 7.7 million English sentences from the European Parliament's proceedings, used for machine translation and multilingual NLP tasks.
- **Parameters:**

| Parameter | Value |
|---|---|
| Batch Size | 32 |
| Total Batches | 20 |
| Maximum Sequence Length | 64 |

- We have evaluate the model on the above mentioned dataset using the perplexity score, which is given by

$$PPL = \exp\left(-\frac{1}{N}\sum_{i=1}^{N}\log P(w_i|w_1, w_2, \ldots, w_{i-1})\right)$$

# Evaluation scores



# Future Work

**1. Optimize KV Cache Implementation**

- Apply **quantization** and **sparse attention** to reduce memory usage.
- Use **LLM Lingua** for KV cache compression via **tensor decomposition** and **clustering**.

**2. Dynamic KV Cache Management**

- Implement **adaptive caching** based on **sequence length** and **model confidence**.

**3. Evaluate on Larger Models and More datasets**

- Analyze **memory-latency-performance trade-offs** on **larger models** and **diverse datasets** (multilingual, domain-specific).
- Study the impact of **sequence length** and **vocabulary size** on KV cache efficiency.

**4. Improving Perplexity and Model Generalization**

- Use **subword tokenization** (BPE, WordPiece, SentencePiece) for better morphology handling.
- Expand model depth with **6-12 layers** and **8-12 attention heads** for enhanced feature extraction.

# Conclusion

**1. KV Cache Improves Efficiency**

- **Significant reduction** in inference latency (55% faster).
- **Substantial savings** in memory usage for self-attention (95% less memory).
- **No degradation** in model performance (perplexity score remains stable).

**2. Practical Implications**

- Enables **faster and more memory-efficient inference** for autoregressive text generation.
- Particularly beneficial for **real-time applications** (e.g., chatbots, translation).

**3. Broader Impact**

- Demonstrates the importance of **optimizing attention mechanisms** in Transformer models.
- Provides a **foundation for future research** on efficient inference techniques.

**4. Final Thoughts**

- The KV cache is a **powerful tool** for improving the efficiency of Transformer decoders.

# Thank you!

The link to the project code is available here:
https://github.com/Rishabh-Thapliyal/ece226-kv-cache.git